

Guide d'accompagnement — TodoList Modern Vanilla JS

EAFC Charlemagne — BES Webdeveloper A2

Pascal Lacroix — ©Pascal Lacroix 2026

Ce guide accompagne la playlist YouTube vidéo par vidéo. À chaque étape, tu trouveras ce qui change, pourquoi on le fait, et vers quoi ça nous emmène. Tu n'as pas besoin de tout comprendre d'un coup — lis l'étape de la vidéo que tu viens de regarder, et reviens quand tu en as besoin.

Avant de commencer — ce qu'on va construire

Une **ToDoList** — une application de gestion de tâches connectée à une API REST.

Elle doit pouvoir :

- afficher les tâches existantes au chargement
- ajouter une tâche
- cocher/décocher une tâche (toggle)
- modifier une tâche
- supprimer une tâche

C'est un **CRUD** : Create, Read, Update, Delete.

Ce qui rend cette version différente de ce que tu as fait en BES 1 : l'application est découpée en **composants** — des morceaux autonomes, chacun responsable d'une seule chose. C'est la façon de travailler dans les frameworks modernes comme Vue.js. Ici on le fait à la main, en Vanilla JS pur — pour comprendre le mécanisme avant que Vue le fasse à ta place.

Vidéo 1 — Création et test de l'API

Ce qu'on fait

On teste l'API avec Insomnia avant d'écrire une seule ligne de code.

Pourquoi

Avant de connecter une application à un serveur, il faut savoir exactement ce que le serveur renvoie et comment lui parler. Insomnia permet de faire ça sans code — on envoie des requêtes et on voit les réponses brutes.

Les 4 requêtes testées correspondent aux 4 opérations du CRUD :

`GET /todos` → lire toutes les tâches

`POST /todos` → créer une tâche

`PUT /todos/:id` → modifier une tâche

`DELETE /todos/:id` → supprimer une tâche

Ce qu'on retient

Le serveur parle en **JSON**. Chaque tâche a :

```
{  
  "id": 1,  
  "content": "Ma tâche",  
  "completed": false,  
  "createdAt": 1234567890  
}
```

Ces propriétés, on les retrouvera partout dans le code.

Vidéo 2 — Principes de la programmation asynchrone

Ce qu'on fait

On comprend pourquoi `async` et `await` sont indispensables dès qu'on parle à un serveur.

Pourquoi

Quand ton application demande des données à une API, elle ne reçoit pas la réponse instantanément. Le réseau prend du temps.

Sans `async/await`, le code continuerait à s'exécuter avant d'avoir reçu la réponse — comme si tu envoyais une commande et que tu essayais de manger avant que la pizza soit livrée.

`await` dit au code : "attends ici que la réponse arrive avant de continuer". `async` dit à la fonction : "cette fonction peut contenir des `await`".

Ce qu'on retient

```
// Sans async/await — le résultat arrive trop tard
const data = fetch("https://..."); // data = une Promise, pas les données

// Avec async/await — on attend la réponse
const response = await fetch("https://...");
const data = await response.json(); // data = les vraies données
```

Dès qu'on parle à une API, toute la chaîne d'appel devient `async`.

Vidéo 3 — Mise en place des composants

D'où on vient

Un projet Vite vide avec juste `main.js`.

Ce qu'on ajoute

La structure des fichiers et les premières briques de l'architecture.

```
□
src/
├── main.js
├── DB.js
├── components/
│   ├── todo/
│   │   ├── Todo.js
│   │   └── template.js
│   └── todoList/
│       ├── TodoList.js
│       └── template.js
```

Pourquoi cette structure

`DB.js` — la couche données. Elle sait parler à l'API. Elle ne sait rien afficher. Elle ne connaît pas le DOM.
Analogie : c'est le facteur — il fait les allers-retours avec le serveur, rien de plus.

`Todo.js` — représente UNE tâche. Il connaît ses propriétés (id, content, completed) et sait se rendre à l'écran. Analogie : c'est la fiche d'une personne dans un carnet d'adresses.

`TodoList.js` — le chef d'orchestre. Il charge les tâches, les affiche, gère les actions (ajout, suppression...).
Analogie : c'est le carnet d'adresses entier — il contient toutes les fiches et décide quoi en faire.

`template.js` (x2) — la mise en forme HTML. Chaque composant a son propre template : une fonction qui reçoit des données et retourne du HTML sous forme de chaîne de caractères.

Le code à cette étape

DB.js

```

js
export default class DB {
  static setApiURL(data) {
    this.apiUrl = data;
  }

  static async findAll() {
    const response = await fetch(this.apiUrl + "todos");
    return response.json();
  }
}
```

`static` signifie qu'on n'a pas besoin d'instancier `DB` avec `new DB()`. On l'utilise directement : `DB.findAll()`. C'est intentionnel — `DB` est un outil unique, pas un objet qu'on duplique.

Todo.js

```

js
import getTemplate from "../template";

export default class Todo {
  constructor(data) {
    this.id = data.id;
    this.content = data.content;
    this.completed = data.completed;
    this.createdAt = data.createdAt;
    this.domElt = null;
  }

  render(el) {
    const template = document.createElement("div");
    template.innerHTML = getTemplate(this);
    el.append(template);
  }
}
```

`constructor(data)` reçoit un objet JSON (venu de l'API) et en extrait les propriétés utiles. `this.domElt = null` — on réserve une place pour stocker le nœud DOM plus tard.

TodoList.js

```

□
js
import DB from "../DB";
import Todo from "../todo/Todo";
import getTemplate from "../template";

export default class TodoList {
  constructor(data) {
    this.domElt = document.querySelector(data.el);
    DB.setApiURL(data.apiUrl);
    this.todos = [];
    this.loadTodos();
  }

  async loadTodos() {
    const todos = await DB.findAll();
    this.todos = todos.map((todo) => new Todo(todo));
    this.render();
  }

  render() {
    this.domElt.innerHTML = getTemplate();
    this.todos.forEach((todo) =>
      todo.render(this.domElt.querySelector(".todo-list"))
    );
  }
}

```

`this.todos = todos.map((todo) => new Todo(todo))` — on transforme chaque objet JSON brut de l'API en une vraie instance de `Todo`. C'est la différence entre des données brutes et des objets qui ont des méthodes (comme `render()`).

main.js

```

□
js
import TodoList from "../components/todoList/ToDoList";

new TodoList({
  el: "#app",
  apiUrl: "https://68ad9556a0b85b2f2cf3e1b0.mockapi.io/",
});

```

Le point d'entrée de l'application. Une seule ligne utile : on crée une `TodoList` en lui donnant l'élément DOM cible et l'URL de l'API.

Vidéo 4 — Intégration des templates

D'où on vient

`Todo.render(el)` crée un `div` générique et injecte du HTML dedans. Ce n'est pas propre — le `div` s'intercale dans le DOM entre le `<ul>` et les `<li>`.

Ce qu'on change dans `Todo.js`

```

☐
js
// AVANT
render(el) {
  const template = document.createElement("div");
  template.innerHTML = getTemplate(this);
  el.append(template);
}

// APRÈS
render(el) {
  const template = document.createElement("template");
  template.innerHTML = getTemplate(this);
  this.domElt = template.content.firstChild;
  el.append(this.domElt);
}
```

Pourquoi `document.createElement("template")`

L'élément HTML `<template>` est spécial : son contenu est parsé mais pas rendu. On peut donc y injecter du HTML et en extraire le premier vrai nœud (`firstElementChild`) sans pollution du DOM.

`this.domElt` stocke maintenant une référence directe au `<li>` de cette tâche. On en aura besoin pour le modifier plus tard (toggle, update, delete) sans avoir à le rechercher dans le DOM.

Ce qu'on ajoute dans `todo/template.js`

Le vrai HTML d'une tâche — avec les classes TodoMVC, la checkbox, le label, le bouton destroy et le champ d'édition.

```

☐
js
```

```
export default function getTemplate(todo) {
  return `
    <li data-id="${todo.id}" class="${todo.completed ? "completed" : ""}">
      <div class="view">
        <input class="toggle" type="checkbox"
          ${todo.completed ? "checked" : ""} />
        <label>${todo.content}</label>
        <button class="destroy"></button>
      </div>
      <input type="text" class="edit" value="${todo.content}" />
    </li>
  `;
}
```

`data-id="${todo.id}"` — on stocke l'id sur le nœud DOM. Ça permettra de retrouver la tâche dans `this.todos` par son id.

Vidéo 5 — Items left count

Ce qu'on ajoute dans `TodoList.js`

```

□
js
getItemsLeftCount() {
  return this.todos.filter((todo) => !todo.completed).length;
}

renderItemsLeftCount() {
  this.domEl.querySelector(".todo-count strong").innerText =
    this.getItemsLeftCount();
}
```

Et on appelle `renderItemsLeftCount()` à la fin de `render()`.

Pourquoi deux méthodes séparées

`getItemsLeftCount()` **calcule** — elle ne touche pas au DOM. `renderItemsLeftCount()` **affiche** — elle ne calcule pas.

C'est le principe de séparation des responsabilités : une méthode = une chose. On pourra réutiliser `getItemsLeftCount()` ailleurs sans toucher au DOM, et `renderItemsLeftCount()` sans recalculer manuellement.

`filter((todo) => !todo.completed)` — on garde uniquement les tâches dont `completed` est `false`.
`.length` donne le nombre.

Vidéo 6 — Ajout d'une Todo

Ce qu'on ajoute dans `DB.js`

```
□  
js  
static async create(data) {  
  const response = await fetch(this.apiUrl + "todos", {  
    method: "POST",  
    headers: { "Content-Type": "application/json" },  
    body: JSON.stringify({  
      content: data,  
      completed: false,  
      createdAt: Date.now(),  
    }),  
  });  
  return response.json();  
}
```

`POST` envoie des données au serveur. `JSON.stringify()` convertit l'objet JS en chaîne JSON — c'est le format attendu par l'API. Le serveur renvoie la tâche créée avec son `id` — on en a besoin pour la suite.

Ce qu'on ajoute dans `TodoList.js`

```
□
```

```
js
```

```

async addTodo(data) {
  const todo = await DB.create(data);
  const newTodo = new Todo(todo);
  this.todos.push(newTodo);
  newTodo.render(this.listDomElt);
  this.renderItemsLeftCount();
}

initEvents() {
  this.domElt.querySelector(".new-todo").addEventListener("change", (e) => {
    this.addTodo(e.target.value);
    e.target.value = "";
  });
}

```

Pourquoi cet ordre dans `addTodo()`

D'abord la DB — pour obtenir l'`id` généré par le serveur

Ensuite le tableau — `this.todos.push(newTodo)`

Ensuite le DOM — `newTodo.render(this.listDomElt)`

Ensuite le compteur

On suit toujours le même ordre : DB → tableau → DOM. C'est une convention à respecter pour rester cohérent.

Vidéo 7 — Toggle Completed

Ce qu'on ajoute dans `Todo.js`

```

☐
js
async toggleCompleted() {
  this.completed = !this.completed;
  this.domElt.classList.toggle("completed");
  window.TODOList.renderItemsLeftCount();
  return await DB.updateOne(this);
}

initEvents() {
  this.domElt.querySelector(".toggle").addEventListener("change", () => {
    this.toggleCompleted();
  });
}

```

Et `render(el)` appelle maintenant `initEvents()` avant d'appender.

Ce qu'on ajoute dans `DB.js`

```
□
js
static async updateOne(todo) {
  const response = await fetch(this.apiUrl + "todos/" + todo.id, {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      content: todo.content,
      completed: todo.completed,
    }),
  });
  return response.json();
}
```

Pourquoi `window.TodoList`

`Todo` a besoin de mettre à jour le compteur après un toggle — mais le compteur appartient à `TodoList`. On expose `TodoList` sur `window` pour que `Todo` puisse y accéder.

C'est un **couplage global** — `Todo` connaît son parent par son nom. Ce n'est pas idéal, mais c'est fonctionnel. On reviendra sur ce point dans la vidéo 12 pour faire mieux.

Ce qu'on change dans `main.js`

```
□
js
window.TodoList = new TodoList({
  el: "#app",
  apiUrl: "https://68ad9556a0b85b2f2cf3e1b0.mockapi.io/",
});
```

On expose l'instance sur `window` pour que `Todo` puisse l'appeler.

Vidéo 8 — Delete One By Id

Ce qu'on ajoute dans `DB.js`

```
□
js
```

```
static async deleteOneById(id) {  
  const response = await fetch(this.apiUrl + "todos/" + id, {  
    method: "DELETE",  
  });  
  return response.json();  
}
```

Ce qu'on ajoute dans `TodoList.js`

```
  
js  
async deleteOneById(id) {  
  const resp = await DB.deleteOneById(id);  
  
  this.todos.splice(  
    this.todos.findIndex((todo) => todo.id == id),  
    1  
  );  
  
  this.domElt.querySelector(`[data-id='${id}']`).remove();  
  this.renderItemsLeftCount();  
}
```

Ce qu'on ajoute dans `Todo.initEvents()`

```
  
js  
this.domElt.querySelector(".destroy").addEventListener("click", () => {  
  window.TODOList.deleteOneById(this.id);  
});
```

Pourquoi `findIndex` + `splice`

`findIndex` trouve la position de la tâche dans `this.todos`. `splice(index, 1)` supprime 1 élément à cette position. C'est la façon standard de supprimer un élément d'un tableau quand on connaît son index.

Vidéo 9 — Update One

Ce qu'on ajoute dans `Todo.js`

```
  
js
```

```
async update(data) {  
  this.content = data;  
  this.domEl.querySelector("label").innerText = this.content;  
  this.domEl.classList.remove("editing");  
  return await DB.updateOne(this);  
}
```

Et dans `initEvents()` :

```
☐  
  
js  
this.domEl.querySelector("label").addEventListener("dblclick", () => {  
  this.domEl.classList.add("editing");  
});  
  
this.domEl.querySelector(".edit").addEventListener("change", (e) => {  
  this.update(e.target.value);  
});
```

Pourquoi la classe `editing`

Le CSS TodoMVC utilise `.editing` pour basculer l'affichage :

sans `editing` → on voit le label + les boutons (mode lecture)

avec `editing` → on voit le champ input (mode édition)

Le double-clic active le mode édition. Le `change` sur l'input valide et repasse en mode lecture.

Vidéo 10 — Ajustement des renders

Le problème

`Todo.render(e1)` reçoit l'élément parent et s'y appende lui-même. Ça crée un couplage : `Todo` doit connaître son parent pour se rendre.

Ce qu'on change dans `Todo.js`

```
☐
```

```
js
```

```
// AVANT
render(el) {
  const template = document.createElement("template");
  template.innerHTML = getTemplate(this);
  this.domElt = template.content.firstChild;
  this.initEvents();
  el.append(this.domElt); // ← Todo s'appende lui-même
}

// APRÈS
render() {
  const template = document.createElement("template");
  template.innerHTML = getTemplate(this);
  this.domElt = template.content.firstChild;
  this.initEvents();
  return this.domElt; // ← Todo retourne son nœud, c'est le parent qui appende
}
```

Ce qu'on change dans `TodoList.js`

```
☐
js
// AVANT
this.todos.forEach((todo) => todo.render(this.listDomElt));
newTodo.render(this.listDomElt);

// APRÈS
this.todos.forEach((todo) => this.listDomElt.append(todo.render()));
this.listDomElt.append(newTodo.render());
```

Pourquoi c'est mieux

`Todo` ne connaît plus son parent. Il produit son nœud et le retourne — c'est `TodoList` qui décide où le placer. Chacun son rôle.

C'est le même principe que `getTemplate()` : une fonction qui **produit** quelque chose ne devrait pas décider où ça va.

Vidéo 11 — Émission d'événements vers le parent

Le problème qu'on résout

Dans les vidéos précédentes, `Todo` appelle directement `window.TODOList` pour signaler ce qui s'est passé :

```
☐
js
window.TODOList.renderItemsLeftCount(); // dans toggleCompleted()
window.TODOList.deleteOneById(this.id); // dans initEvents()
```

`TODO` connaît son parent par son nom global. C'est un couplage — si on renomme `TODOList`, si on a plusieurs listes, ça casse.

La solution — les événements personnalisés

Au lieu d'appeler directement son parent, `TODO` **émet un signal**. `TODOList` **écoute** ce signal et décide quoi faire.

C'est exactement ce que fait Vue avec `$emit` — on le réimplémente ici avec l'API native du navigateur.

Ce qu'on change dans `TODO.js`

On ajoute une méthode `dispatch()` :

```
☐
js
dispatch(type, detail) {
  this.domElt.dispatchEvent(
    new CustomEvent(type, { bubbles: true, detail })
  );
}
```

`CustomEvent` crée un événement personnalisé avec un nom (`type`) et des données (`detail`). `bubbles: true` — l'événement remonte dans l'arbre DOM jusqu'à ce que quelqu'un l'intercepte.

On remplace les appels `window.TODOList.*` par des émissions :

```
☐
js
async toggleCompleted() {
  this.completed = !this.completed;
  this.domElt.classList.toggle("completed");
  this.dispatch("todo:updated", { todo: this }); // ← signal vers le parent
}

async update(data) {
  this.content = data;
  this.domElt.querySelector("label").innerText = this.content;
  this.domElt.classList.remove("editing");
  this.dispatch("todo:updated", { todo: this }); // ← signal vers le parent
}
```

```

☐
js
this.domElt.querySelector(".destroy").addEventListener("click", () => {
  this.dispatch("todo:deleted", { id: this.id }); // ← signal vers le parent
});

```

On supprime aussi `import DB from "../..DB"` — `Todo` ne parle plus directement à la base de données.

Ce qu'on change dans `TodoList.js`

On ajoute `updateOne()` — c'est maintenant `TodoList` qui gère la DB :

```

☐
js
async updateOne(todo) {
  this.renderItemsLeftCount();
  return await DB.updateOne(todo);
}

```

Et dans `initEvents()`, on écoute les événements qui remontent des Todo :

```

☐
js
this.listDomElt.addEventListener("todo:updated", async (e) => {
  await this.updateOne(e.detail.todo);
});

this.listDomElt.addEventListener("todo:deleted", async (e) => {
  await this.deleteOneById(e.detail.id);
});

```

Ce qu'on change dans `main.js`

```

☐
js
// AVANT
window.TODOList = new TODOList({ ... });

// APRÈS
new TODOList({ ... });

```

`window.TODOList` disparaît — on n'en a plus besoin.

Le parallèle avec Vue

Ce qu'on vient de faire	Ce que Vue fait
<code>this.dispatch("todo:deleted", { id })</code>	<code>this.\$emit("deleted", id)</code>
<code>addEventListener("todo:deleted", ...)</code>	<code>@deleted="deleteOneById"</code> sur le composant parent
<code>bubbles: true</code>	implicite dans le système de Vue
<code>Todo</code> ne sait plus qui est son parent. Il crie dans le vide : "quelque chose s'est passé !" et c'est <code>TodoList</code> qui décide quoi faire. C'est ça, le vrai découplage.	

État final du projet



Qui fait quoi

Fichier	Responsabilité
<code>main.js</code>	Démarrage — instancie <code>TodoList</code>
<code>DB.js</code>	Parle à l'API — rien d'autre
<code>Todo.js</code>	Représente une tâche, gère ses propres événements
<code>todo/template.js</code>	HTML d'une tâche
<code>TodoList.js</code>	Orchestre tout : tableau, DOM, DB
<code>todoList/template.js</code>	HTML de l'application

La chaîne de responsabilités



Utilisateur clique "destroy"

→ Todo émet "todo:deleted"

→ l'événement bulle vers TodoList

→ TodoList.deleteOneById()

→ DB.deleteOneById() (serveur)

→ this.todos.splice() (tableau)

→ this.domElt.remove() (DOM)

→ renderItemLeftCount() (compteur)

Chaque acteur fait sa partie. Personne ne fait le travail d'un autre.